

ESTUDO DE CASO / PROJETOS DE DADOS

Do S3 ao OCI: entrega de dados por cliente e operação de pipelines

Um relato técnico sobre distribuição de arquivos de Data Quality, validação de dados, processamento em cloud e rotinas que precisam continuar funcionando depois da primeira execução.

Alexandre Roberto · São Paulo · Setembro de 2026

Os arquivos de Data Quality eram disponibilizados diariamente no Amazon S3, separados por cliente. Para chegar ao destino, precisavam atravessar um processo orquestrado no Azure Data Factory, passar por uma área intermediária no Azure Blob Storage e ser entregues no malote correspondente no OCI Object Storage. O destino não era uma constante do pipeline: dependia do código do cliente, consultado em uma base Azure SQL. Esse detalhe muda o problema de “copiar arquivos” para descobrir entradas, resolver destinos e tratar exceções sem perder o restante do lote.

Este é o projeto mais detalhado do documento. Na segunda parte, descrevo outras cinco situações em que trabalhei com qualidade, processamento distribuído, eventos, automação de incidentes, reprocessamento e BI. Em cada uma, apresento o fluxo e o motivo técnico das escolhas que fizeram parte do trabalho.

1. O problema: uma entrega diária com destino variável

A referência de processamento era D-1. A execução diária precisava localizar os arquivos desse período no S3, identificar a quais clientes pertenciam e procurar o malote de cada um. Se a consulta não encontrasse um destino cadastrado, não faria sentido mover o arquivo para um lugar presumido. A ocorrência era registrada e o fluxo continuava com os outros clientes.

Essa separação por cliente evitava que uma inconsistência de cadastro interrompesse as demais entregas. O fluxo tinha logs, alertas, retries para falhas de comunicação e reprocessamento manual por data. Retry cobre falhas transitórias; a repetição por data permite retomar um lote depois de corrigida a causa.

2. Arquitetura do fluxo

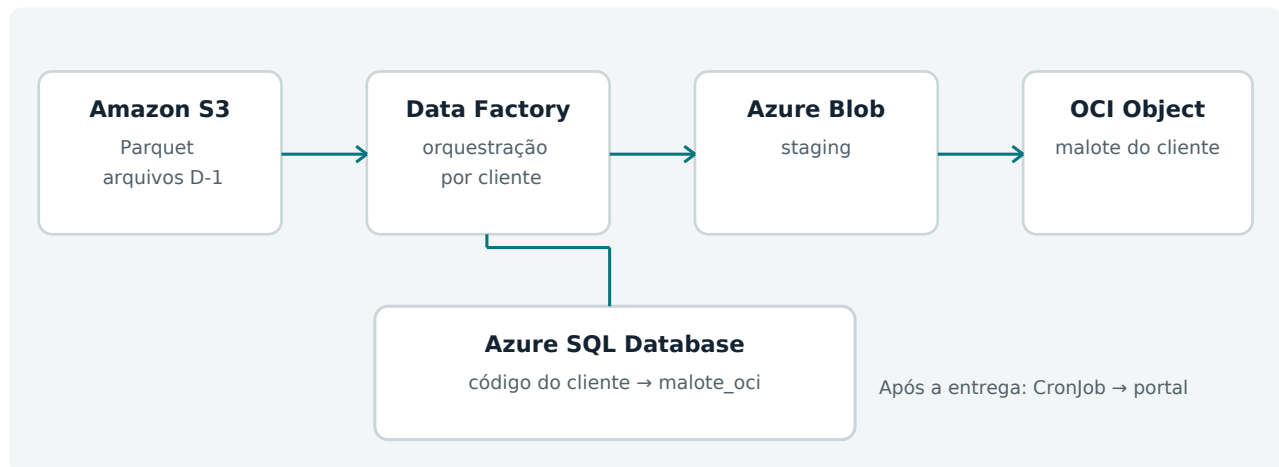


Figura 1 — Fluxo funcional dos arquivos. O Azure SQL participa da decisão de roteamento; a publicação no portal ocorre depois da movimentação entre as nuvens.

O Azure Data Factory coordena a execução. Seu conector Amazon S3 oferece leitura como origem em Copy Activity e suporte a Get Metadata; o conector Azure Blob permite a etapa intermediária de cópia. A documentação dos conectores confirma essas capacidades, enquanto o uso delas neste caso vem do desenho do projeto. [\[1\]\[2\]\[6\]](#)

Descoberta dos arquivos e cálculo da data

O Schedule Trigger inicia o pipeline diariamente. No fluxo, a data de referência é calculada como D-1 e utilizada na busca dos arquivos daquele ciclo. Get Metadata obtém informações sobre itens existentes no caminho configurado, permitindo levantar o que efetivamente chegou antes de movimentar dados. [\[2\]\[5\]](#) Uma execução agendada, por si só, não significa que a entrada está completa; por isso a descoberta é uma etapa separada da cópia.

A partir dos arquivos encontrados, o processo identifica os clientes presentes no lote. O ForEach executa a sequência de atividades para cada cliente. A iteração por destinatário permite resolver o malote, decidir se a cópia pode seguir e registrar o resultado individualmente. A documentação da atividade descreve os modos sequencial e paralelo. [\[3\]](#)

Roteamento no Azure SQL

Para cada código de cliente, uma atividade Lookup consulta o Azure SQL Database e retorna o valor de malote_oci. Antes de movimentar o arquivo, o fluxo valida a resposta da consulta. Quando não existe correspondência, registra a pendência e segue para o próximo cliente. A documentação do Lookup explica o retorno de uma linha ou de uma coleção; a regra de negócio deste caso é o vínculo entre cliente e malote. [\[4\]](#)

Essa decisão não deve ser confundida com transformação de dados. O arquivo Parquet conserva seu papel de artefato a ser entregue; a consulta ao banco resolve para onde ele deve seguir. Colocar o mapeamento fora de um endereço fixo no pipeline torna possível tratar clientes distintos no mesmo ciclo sem criar uma cópia estática do fluxo para cada um.

Staging no Azure e entrega em OCI

Depois da validação, os arquivos passam pelo Azure Blob Storage antes de chegar ao OCI Object Storage. No projeto, o Blob funcionou como staging entre as plataformas. O Object Storage organiza os dados em buckets e objetos; “malote” é a designação funcional do destino atribuído ao cliente. [6][7] O mecanismo exato de autenticação e transferência do Blob para OCI não foi incluído no material de origem, então o diagrama representa a sequência funcional sem sugerir um conector específico.

Há uma consequência operacional nessa divisão: localizar um arquivo no Blob não equivale a confirmar a entrega no OCI. São etapas com estados diferentes, que exigem observação própria quando ocorre uma falha. O monitoramento e os alertas do fluxo serviam para tornar essas exceções visíveis durante a execução.

Publicação no portal

A disponibilização para consulta tinha ainda uma etapa posterior. Um CronJob no Kubernetes, executado a cada 30 minutos, acionava uma rotina de cópia dos arquivos prontos da área de staging para o sistema de arquivos compartilhado pelo portal. O CronJob é o recurso do Kubernetes para criar Jobs em uma programação recorrente. [8] Nesse desenho, entregar um objeto e torná-lo visível ao usuário são eventos separados.

3. Operar o fluxo: exceções e retomada

O caso que mais demonstra a necessidade de tratamento explícito é o cliente sem malote cadastrado. Não é uma falha genérica de infraestrutura: falta uma decisão de roteamento. Registrar a ocorrência e continuar o lote preserva as entregas que têm destino válido e deixa uma pendência concreta para correção. Já uma falha de comunicação entre plataformas admite uma política de retry. Misturar as duas situações produziria tentativas inúteis para um problema de cadastro.

O reprocessamento manual por data permitia repetir um ciclo específico após a investigação. A data D-1, portanto, não é só um cálculo do agendamento: ela também é a referência que o operador pode informar ao retomar um lote. Na análise de uma ocorrência, as perguntas são concretas: o arquivo chegou? O cliente tinha malote? A cópia terminou no Blob? A entrega no OCI foi concluída? O portal recebeu o arquivo? Cada resposta aponta para uma etapa e uma ação diferente.

A solução automatizou a distribuição diária por cliente e manteve caminhos para tratar destinatários não mapeados, acompanhar exceções e retomar uma data específica.

4. Qualidade de dados antes da integração

Em outro fluxo, os arquivos chegavam ao Data Lake e passavam por validações antes de serem liberados para integração. O trabalho combinava Python, SQL, PySpark/Spark, Amazon S3, EMR e EMR Serverless. O uso de Parquet estava ligado ao armazenamento dos arquivos; o formato colunar favorece leituras analíticas, mas não substitui verificação de conteúdo. [9]

As regras cobriam esquema e layout, campos obrigatórios, tipos, formato, nulidade, cardinalidade e dependências entre conjuntos de dados. Há diferenças importantes entre essas falhas. Um campo ausente pode impedir que o arquivo seja interpretado conforme o contrato esperado; um campo presente com valor inválido só aparece após a leitura; uma dependência ausente pode exigir que outro conjunto chegue ou seja corrigido. O processamento identificava inconsistências e permitia que os registros aprovados seguissem para a próxima etapa.

Python e SQL davam forma às regras de tratamento e consulta, enquanto Spark/PySpark era empregado nas transformações distribuídas. EMR Serverless executa jobs Spark sem exigir administração direta de clusters pelo usuário. [10] Para a sustentação, a distinção mais útil é entre um job que falhou, um arquivo rejeitado por estrutura e registros isolados como inconsistentes. São diagnósticos e ações de correção diferentes.

5. Quando o arquivo inicia o processamento

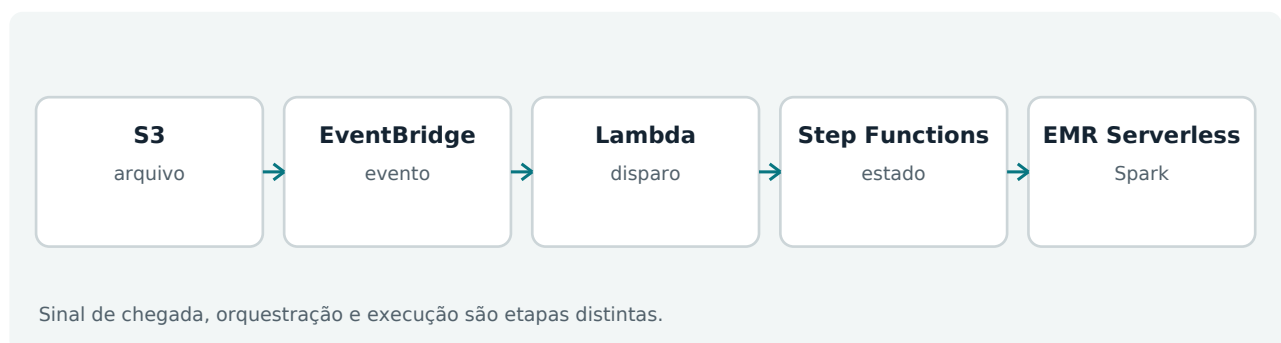


Figura 2 — Encadeamento relatado para o pipeline orientado a eventos. O evento aciona o fluxo; o processamento é executado pelo job Spark.

O terceiro projeto liga a chegada de arquivos ao processamento: S3 envia o evento ao EventBridge, Lambda inicia ou encaminha a execução, Step Functions coordena as etapas e EMR Serverless executa rotinas em Python/Spark. Parâmetros atravessam as etapas, e falhas de execução são acompanhadas pela equipe. A AWS documenta notificações S3, Retry e Catch no Step Functions e logs para os jobs do EMR Serverless. [11][12][13]

Uma notificação não deve ser tomada como prova de que o dado está pronto. Ela informa um acontecimento no S3; a máquina de estados e o job ainda precisam terminar. Além disso, a AWS descreve as notificações do S3 como entregues pelo menos uma vez, sem garantia de ordem. [11] Repetição e ordem são, portanto, pontos que merecem atenção ao revisar esse tipo de arquitetura.

6. Transformar falhas em incidentes

A automação de incidentes nasceu de um problema percebido durante uma Daily: quando uma Step Function falhava, identificar a exceção e abrir um chamado no Helix ainda dependia de intervenção manual. Desenvolvi uma prova de conceito em Python para capturar as informações da falha e chamar a API REST do BMC Helix por uma conta de serviço. A POC evoluiu até ser utilizada em produção.

O valor técnico dessa integração está na passagem de contexto entre duas operações. O Step Functions conhece a execução e o erro; o Helix organiza o atendimento do incidente. A API da BMC documenta a criação de registros, incluindo incidentes, por requisição POST. [14] A automação passou a abrir o chamado com dados relacionados à falha, retirando essa ação da rotina manual da equipe.

7. Reprocessamento no ciclo de seguro garantia

O caso de seguro garantia partia de apólices e endossos que não haviam sido integrados inicialmente por informações incompletas ou inconsistentes enviadas pelos clientes. Uma rotina em Shell baixava os arquivos D-1 do servidor. Com Python e SQL, o processo examinava os status de integração e selecionava os números de apólice que precisavam retornar ao fluxo.

O próximo artefato era um arquivo .sql gerado para a limpeza e carga de uma tabela de reprocessamento no Oracle. Esse arquivo seguia anexado a um chamado no Helix para execução pelo suporte à produção dentro do SLA. Após essa preparação, um processo noturno em Spring Batch realizava o reprocessamento. A documentação do Spring Batch descreve a configuração e as propriedades dos jobs. [15]

A divisão do trabalho é importante. Localizar um registro que falhou não executa a correção; gerar o SQL não equivale a aplicá-lo em produção; e reapresentar um registro ao job não garante, por si só, que a integração final foi aceita. O chamado e o arquivo anexo formavam a passagem para o suporte, preservando a seleção feita pela rotina e a ação que precisava ser executada. Esse fluxo mostra engenharia de dados em uma situação de sustentação, com intervenção controlada e dependência de um ciclo batch.

8. Da consolidação ao fechamento mensal

O sexto projeto reuniu dados de Oracle, SQL Server, APIs e arquivos CSV/TXT para apoiar um fechamento mensal. Consultas e transformações SQL consolidavam informações de áreas distintas; a camada analítica era organizada para consumo no Power BI. O resultado precisava apoiar a apresentação no primeiro dia útil do mês seguinte. O trabalho combinava integração de fontes, Data Warehouse, modelagem dimensional e leitura das regras de negócio.

Em um modelo dimensional, uma tabela fato representa observações em uma granularidade definida; as dimensões dão contexto para filtrar e agrupar essas observações. A orientação da Microsoft para Power BI explica a utilidade do esquema em estrela para modelos semânticos. [16] Essa referência ajuda a entender por que separar fatos e dimensões fazia sentido no projeto. Ela não descreve as tabelas específicas que existiam no ambiente.

O prazo de fechamento também impõe uma pergunta técnica: quais dados e quais regras pertencem ao período apresentado? Se fontes diferentes chegam em ritmos distintos, o total de um indicador só é defensável quando a extração, a transformação e o recorte temporal são coerentes. Era essa ligação entre preparação técnica e apresentação dos resultados que o processo de fechamento precisava sustentar.

Conclusão

Os seis projetos têm ambientes diferentes, mas uma característica comum: havia uma passagem de responsabilidade entre sistemas ou equipes. Um arquivo sai do S3 e precisa encontrar o cliente correto; um registro só avança depois de validado; um evento precisa chegar até o job; uma falha precisa virar incidente; uma apólice precisa retornar ao batch; um resultado precisa ser compreendido no fechamento. Minha atuação nesses fluxos reúne SQL, Python, BI, integração, qualidade e sustentação de dados.

Esse é o tipo de trabalho que posso assumir por projeto, contrato temporário ou reforço de equipe: entender a entrada e a regra, construir ou ajustar o processamento e deixar a operação com condições de identificar o que aconteceu quando algo foge do previsto.

Fontes e escopo

Os fatos sobre os projetos vêm do meu material profissional e do detalhe sobre a publicação no portal acrescentado posteriormente. As referências abaixo explicam capacidades e comportamento das tecnologias; não comprovam configurações internas, volume ou resultados quantitativos destes ambientes. Diagramas são representações funcionais e omitem nomes de clientes, segredos e topologia interna. Consulta às fontes: 24 set. 2026.

- [1] Microsoft Learn. [Copy and transform data in Amazon S3 with Azure Data Factory](#).
- [2] Microsoft Learn. [Get Metadata activity](#).
- [3] Microsoft Learn. [ForEach activity](#).
- [4] Microsoft Learn. [Lookup activity](#).
- [5] Microsoft Learn. [Create schedule triggers](#).
- [6] Microsoft Learn. [Copy and transform data in Azure Blob Storage](#).
- [7] Oracle. [Overview of OCI Object Storage](#).
- [8] Kubernetes. [CronJob](#).
- [9] Apache Parquet. [Overview](#).
- [10] Amazon Web Services. [Using Spark configurations with EMR Serverless jobs](#).
- [11] Amazon Web Services. [Event notification types and destinations in Amazon S3](#).
- [12] Amazon Web Services. [Error handling in AWS Step Functions](#).
- [13] Amazon Web Services. [Logging for EMR Serverless](#).
- [14] BMC. [BMC Helix ITSM platform REST API references](#).
- [15] Spring. [Spring Batch: configuring a job](#).
- [16] Microsoft Learn. [Understand star schema and the importance for Power BI](#).